Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №3
По дисциплине: “СПП”

Выполнил:
Студент 3 курса
Группы ПО-12
Мартынюк В. В.
Проверила:
Кулик А. Д.

Брест 2026

Цель работы: приобрести навыки применения паттернов проектирования при решении практических задач с использованием языка Python.

Задание 1

10) Заводы по производству автомобилей. Реализовать возможность создавать автомобили различных типов на различных заводах.
from abc import ABC, abstractmethod

```
class Car:
    def __init__(self, brand: str, model: str, body_type: str, engine: str):
        self.brand = brand
        self.model = model
        self.body_type = body_type
        self.engine = engine
        self.options = []

    def add_option(self, option: str):
        self.options.append(option)

    def get_info(self) -> str:
        options = f", options: {', '.join(self.options)}" if self.options else ""
        return f"{self.brand} {self.model} ({self.body_type}) - {self.engine}{options}"


class Factory(ABC):
    @abstractmethod
    def create_car(self, body_type: str, model: str) -> Car:
        pass


class ToyotaFactory(Factory):
    def create_car(self, body_type: str, model: str) -> Car:
        specs = {
            "sedan": {"engine": "2.5L Hybrid", "options": ["Safety Sense", "Climate control"]},
            "suv": {"engine": "3.5L V6", "options": ["4WD", "360 camera"]},
            "hatchback": {"engine": "1.8L", "options": ["Park sensors"]},
            "sports": {"engine": "3.0L Twin-Turbo", "options":
                ["Sport suspension", "Carbon brakes"]},
        }

        spec = specs.get(body_type.lower(), {"engine": "2.0L", "options": []})
        car = Car("Toyota", model, body_type, spec["engine"])

        for option in spec["options"]:
            car.add_option(option)

        return car


class BMWFactory(Factory):
    def create_car(self, body_type: str, model: str) -> Car:
        specs = {
            "sedan": {"engine": "2.0L TwinPower Turbo", "options": ["xDrive", "Laser lights"]},
            "suv": {"engine": "4.4L V8", "options": ["Air suspension", "Off-road package"]},
            "hatchback": {"engine": "1.5L", "options": ["Panoramic roof"]},
            "sports": {"engine": "4.0L V8", "options": ["M Performance", "Carbon body"]},
        }

        spec = specs.get(body_type.lower(), {"engine": "2.0L", "options": []})
        car = Car("BMW", model, body_type, spec["engine"])
```

```python
        for option in spec["options"]:
            car.add_option(option)

        return car


class TeslaFactory(Factory):
    def create_car(self, body_type: str, model: str) -> Car:
        specs = {
            "sedan": {"engine": "Dual Motor Electric", "options": ["Autopilot", "Glass roof"]},
            "suv": {"engine": "Plaid Electric", "options": ["Falcon doors", "Bio defense"]},
            "hatchback": {"engine": "Electric", "options": ["Minimalist interior"]},
            "sports": {"engine": "Roadster Electric", "options": ["SpaceX package", "Insane mode"]},
        }

        spec = specs.get(body_type.lower(), {"engine": "Electric", "options": []})
        car = Car("Tesla", model, body_type, spec["engine"])

        for option in spec["options"]:
            car.add_option(option)

        return car


class Dealership:
    def __init__(self, factory: Factory):
        self.factory = factory
        self.inventory = []

    def order_car(self, body_type: str, model: str) -> Car:
        car = self.factory.create_car(body_type, model)
        self.inventory.append(car)
        return car

    def show_inventory(self):
        for car in self.inventory:
            print(car.get_info())


if __name__ == "__main__":
    toyota_dealer = Dealership(ToyotaFactory())
    bmw_dealer = Dealership(BMWFactory())

    toyota_dealer.order_car("sedan", "Camry")
    toyota_dealer.order_car("suv", "Highlander")
    bmw_dealer.order_car("sedan", "3 Series")
    bmw_dealer.order_car("sports", "M4")

    toyota_dealer.show_inventory()
    bmw_dealer.show_inventory()
```

Решение:

```
Toyota Camry (sedan) - 2.5L Hybrid, options: Safety Sense, Climate control
Toyota Highlander (suv) - 3.5L V6, options: 4WD, 360 camera
BMW 3 Series (sedan) - 2.0L TwinPower Turbo, options: xDrive, Laser lights
BMW M4 (sports) - 4.0L V8, options: M Performance, Carbon body
```

Задание 2

10) Система Городской транспорт. На Маршрут назначаются Автобус или Троллейбус. Транспортные средства должны двигаться с определенным для каждого Маршрута интервалом. При поломке на Маршрут должен выходить резервный транспорт или увеличиваться интервал движения.

Решение:

```
from abc import ABC, abstractmethod


class Level(ABC):
    @abstractmethod
    def desc(self) -> str:
        pass

    @abstractmethod
    def discount(self) -> float:
        pass

    @abstractmethod
    def preorder(self) -> bool:
        pass

    @abstractmethod
    def exclusive(self) -> bool:
        pass


class Base(Level):
    def desc(self) -> str:
        return "Base"

    def discount(self) -> float:
        return 0

    def preorder(self) -> bool:
        return False

    def exclusive(self) -> bool:
        return False


class Silver(Level):
    def desc(self) -> str:
        return "Silver"

    def discount(self) -> float:
        return 0.05

    def preorder(self) -> bool:
        return True

    def exclusive(self) -> bool:
        return False


class Gold(Level):
```

```
    def desc(self) -> str:
        return "Gold"

    def discount(self) -> float:
        return 0.1

    def preorder(self) -> bool:
        return True

    def exclusive(self) -> bool:
        return True


class Platinum(Level):
    def desc(self) -> str:
        return "Platinum"

    def discount(self) -> float:
        return 0.15

    def preorder(self) -> bool:
        return True

    def exclusive(self) -> bool:
        return True


class User:
    def __init__(self, name: str):
        self.name = name
        self.spent = 0
        self.level = Base()

    def buy(self, amount: float):
        self.spent += amount
        self.update_level()

    def update_level(self):
        if self.spent >= 50000:
            self.level = Platinum()
        elif self.spent >= 20000:
            self.level = Gold()
        elif self.spent >= 5000:
            self.level = Silver()

    def show(self):
        print(f"{self.name}: {self.level.desc()}")
        print(f"  Discount: {self.level.discount() * 100}%")
        print(f"  Preorder: {self.level.preorder()}")
        print(f"  Exclusive: {self.level.exclusive()}")


if __name__ == "__main__":
    user = User("John")
    user.show()

    user.buy(6000)
    user.show()

    user.buy(20000)
    user.show()
```

```
    user.buy(30000)
    user.show()
```

```
John: Base
  Discount: 0%
  Preorder: False
  Exclusive: False
John: Silver
  Discount: 5.0%
  Preorder: True
  Exclusive: False
```

Задание 3

10) Проект «Банкомат». Предусмотреть выполнение основных операций (ввод пин-кода, снятие суммы, завершение работы) и наличие различных режимов работы (ожидание, аутентификация, выполнение операции, блокировка – если нет денег). Атрибуты: общая сумма денег в банкомате, ID.

```
from abc import ABC, abstractmethod


class ATMState(ABC):
    def __init__(self, atm):
        self.atm = atm

    @abstractmethod
    def insert_card(self):
        pass

    @abstractmethod
    def enter_pin(self, pin):
        pass

    @abstractmethod
    def withdraw(self, amount):
        pass

    @abstractmethod
    def eject_card(self):
        pass


class IdleState(ATMState):
    def insert_card(self):
        print("Карта вставлена")
        self.atm.state = self.atm.card_inserted

    def enter_pin(self, pin):
```

```
        print("Сначала вставьте карту")

    def withdraw(self, amount):
        print("Сначала вставьте карту")

    def eject_card(self):
        print("Нет карты")


class CardInsertedState(ATMState):
    def insert_card(self):
        print("Карта уже вставлена")

    def enter_pin(self, pin):
        if pin == "1234":
            print("Пин принят")
            self.atm.state = self.atm.authenticated
        else:
            print("Неверный пин")
            self.atm.attempts += 1
            if self.atm.attempts >= 3:
                print("Карта заблокирована")
                self.atm.state = self.atm.blocked

    def withdraw(self, amount):
        print("Введите пин")

    def eject_card(self):
        print("Карта извлечена")
        self.atm.state = self.atm.idle
        self.atm.attempts = 0


class AuthenticatedState(ATMState):
    def insert_card(self):
        print("Карта уже вставлена")

    def enter_pin(self, pin):
        print("Пин уже введен")

    def withdraw(self, amount):
        if amount > self.atm.balance:
            print(f"Недостаточно. Доступно: {self.atm.balance}")
            return

        if amount > self.atm.daily_limit:
            print(f"Лимит {self.atm.daily_limit}")
            return
```

```
        self.atm.balance -= amount
        print(f"Выдано {amount}. Остаток: {self.atm.balance}")

        if self.atm.balance == 0:
            print("Деньги закончились")
            self.atm.state = self.atm.empty

    def eject_card(self):
        print("Карта извлечена")
        self.atm.state = self.atm.idle
        self.atm.attempts = 0


class EmptyState(ATMState):
    def insert_card(self):
        print("Нет денег")

    def enter_pin(self, pin):
        print("Нет денег")

    def withdraw(self, amount):
        print("Нет денег")

    def eject_card(self):
        print("Карта извлечена")
        self.atm.state = self.atm.idle


class BlockedState(ATMState):
    def insert_card(self):
        print("Карта заблокирована")

    def enter_pin(self, pin):
        print("Карта заблокирована")

    def withdraw(self, amount):
        print("Карта заблокирована")

    def eject_card(self):
        print("Извлечение невозможно")


class ATM:
    def __init__(self, id, balance):
        self.id = id
        self.balance = balance
        self.daily_limit = 10000
        self.attempts = 0
```

```
        self.idle = IdleState(self)
        self.card_inserted = CardInsertedState(self)
        self.authenticated = AuthenticatedState(self)
        self.empty = EmptyState(self)
        self.blocked = BlockedState(self)

        self.state = self.idle

    def insert_card(self):
        self.state.insert_card()

    def enter_pin(self, pin):
        self.state.enter_pin(pin)

    def withdraw(self, amount):
        self.state.withdraw(amount)

    def eject_card(self):
        self.state.eject_card()

    def status(self):
        print(f"\nATM {self.id}")
        print(f"State: {self.state.__class__.__name__}")
        print(f"Balance: {self.balance}")
        print(f"Attempts: {self.attempts}")


if __name__ == "__main__":
    atm = ATM("ATM001", 5000)

    print("Сценарий 1: Успешная операция")
    atm.insert_card()
    atm.enter_pin("1234")
    atm.withdraw(2000)
    atm.eject_card()

    print("\nСценарий 2: Неверный пин")
    atm.insert_card()
    atm.enter_pin("0000")
    atm.enter_pin("1111")
    atm.enter_pin("2222")
    atm.insert_card()

    print("\nСценарий 3: Снятие больше чем есть")
    atm2 = ATM("ATM002", 1000)
    atm2.insert_card()
    atm2.enter_pin("1234")
    atm2.withdraw(2000)
    atm2.withdraw(500)
```

```
atm2.eject_card()

print("\n Сценарий 4: Пустой банкомат")
atm3 = ATM("ATM003", 500)
atm3.insert_card()
atm3.enter_pin("1234")
atm3.withdraw(500)
atm3.insert_card()
atm3.enter_pin("1234")
```

```
Сценарий 3: Снятие больше чем есть
Карта вставлена
Пин принят
Недостаточно. Доступно: 1000
Выдано 500. Остаток: 500
Карта извлечена

 Сценарий 4: Пустой банкомат
Карта вставлена
Пин принят
Выдано 500. Остаток: 0
Деньги закончились
Нет денег
Нет денег
```

Вывод: В результате выполнения лабораторной работы были закреплены знания объектно-ориентированного программирования Python при решении задач.